

3. Data Analytics II

1. Implement logistic regression using Python/R to perform classification on “winequality-red” dataset and predict whether a particular wine is good or bad based on features.
2. Compute Confusion matrix to find TP, FP, TN, FN, Accuracy, Error rate, Precision, Recall on the given dataset.

```
# Import necessary libraries
```

```
import pandas as pd
```

```
import numpy as np
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.linear_model import LogisticRegression
```

```
from sklearn.metrics import confusion_matrix, accuracy_score, precision_score, recall_score, f1_score
```

```
# Step 1: Load the winequality-red dataset
```

```
url = "https://archive.ics.uci.edu/ml/machine-learning-databases/00374/winequality-red.csv" #  
UCI repository URL
```

```
df = pd.read_csv(url, delimiter=";")
```

```
# Step 2: Explore the dataset
```

```
print("Dataset Preview:")
```

```
print(df.head())
```

```
# Step 3: Convert wine quality to binary (Good vs Bad wine)
```

```
# Here we define "Good" wine as quality >= 7, else "Bad" wine
```

```
df['quality_binary'] = df['quality'].apply(lambda x: 1 if x >= 7 else 0)
```

```
# Step 4: Define predictor variables (X) and target variable (y)
```

```
X = df.drop(columns=['quality', 'quality_binary'])
```

```
y = df['quality_binary']
```

```
# Step 5: Split the dataset into training and testing sets (80% training, 20% testing)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Step 6: Train the Logistic Regression model
logreg = LogisticRegression(max_iter=1000)
logreg.fit(X_train, y_train)

# Step 7: Predict the target variable on the test set
y_pred = logreg.predict(X_test)

# Step 8: Compute Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
print("\nConfusion Matrix:")
print(cm)

# Step 9: Calculate Accuracy, Error Rate, Precision, Recall
accuracy = accuracy_score(y_test, y_pred)
error_rate = 1 - accuracy
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)

print(f"\nModel Evaluation Metrics:")
print(f"Accuracy: {accuracy:.2f}")
print(f"Error Rate: {error_rate:.2f}")
print(f"Precision: {precision:.2f}")
print(f"Recall: {recall:.2f}")
print(f"F1 Score: {f1:.2f}")
```